Quantification du conflit entre mémoire paramétrique et mémoire contextuelle : Étude des Métriques de Dégradation sur la Famille de modèles Qwen3.5

Maximilien Lyonnais[1*] Clément Géliot[1*] Hugo De Bosschere[1*]
Loria, 2 rue Édouard Belin, 57070 Metz, France
hugo.debosschere@student-cs.fr

RÉSUMÉ

Le RAG est une technique consistant à extraire des documents depuis une base de données pour les injecter dans le contexte du modèle. Les LLMs s'appuient de plus en plus sur du RAG pour exploiter des informations récentes ou spécifiques. Cette approche est néanmoins source de conflit entre la *mémoire paramétrique*(le savoir figé dans les poids du modèle) et la *mémoire contextuelle* (les données injectées dans le prompt via le RAG). Un conflit émerge lorsque ces deux mémoires se contredisent : comment le modèle choisit-il alors la source d'information à privilégier ? Nous étudions ici ce conflit via la mise à jour d'APIs. Nous simulons une situation où un LLM, entraîné sur une version antérieure de `NumPy`, reçoit en contexte la documentation de sa nouvelle version. Nous quantifions dans cet article la propension des modèles de la famille qwen3.5 à rejeter leur mémoire paramétrique au profit de la mémoire contextuelle.

ABSTRACT

Quantifying the conflict between parametric and contextual memory: Degradation Metrics Study on the Qwen3.5 Model Family

RAG is a technique that retrieves documents from a database to inject them in the model's context. LLMs increasingly rely on RAG to use recent or specific information. This creates a conflict between the *parametric memory* (knowledge stored in the model's weights) and the *contextual memory* (knowledge injected in the prompt through RAG). When these two sources of information clashes, this creates a conflict : how will then the model choose which information to prefer ? We study here this conflict through API's updates. We simulate a situation where an LLM trained on a previous version of `NumPy` receives in its context the documentation of the new version. We quantify in this article the tendency of qwen3.5 models to choose their contextual memory over their parametric memory.

MOTS-CLÉS : Retrieval-Augmented Generation, RAG, Grands Modèles de Langage, LLM, Mémoire Paramétrique, Mémoire Contextuelle, Ingénierie Logicielle, Évaluation.

KEYWORDS: Retrieval-Augmented Generation, RAG, Large Language Models, LLM, Parametric Memory, Contextual Memory, Software Engineering, Evaluation.

*. Contributions égales

1 Introduction

Les LLMs ont démontré des capacités remarquables, allant de la compréhension du langage naturel à la génération de code complexe. Cependant, ces modèles souffrent de limitations inhérentes à leur conception : ils sont sujets aux hallucinations et demeurent strictement contraints par les connaissances figées dans leur corpus d'entraînement. Ils sont ainsi incapables, par nature, de répondre de manière fiable à des requêtes portant sur des informations récentes ou privées.

Pour pallier ce problème, la technique du *Retrieval-Augmented Generation* (RAG) s'est imposée comme un standard (Lewis *et al.*, 2020). Le RAG consiste à extraire des documents pertinents depuis une base de données externe pour les injecter directement dans la fenêtre de contexte du modèle avant l'étape de génération.

Cette nouvelle technologie met néanmoins en lumière une tension fondamentale au cœur de l'architecture des LLMs, partagés entre deux sources de connaissances :

— **La mémoire paramétrique** : le savoir encodé de manière statique dans les poids du réseau de neurones lors de sa phase d'entraînement.

— **La mémoire contextuelle** : les informations éphémères injectées directement dans le prompt lors de l'inférence.

Un conflit majeur émerge lorsque ces deux mémoires se contredisent. Comment le modèle choisit-il la source d'information à privilégier ? Longpre et al. (Longpre *et al.*, 2021) ont démontré que les LLMs affichent une forte préférence pour leurs données d'entraînement, même face à des faits substitués en contexte. Xie et al. (Xie *et al.*, 2023) ont montré que cette susceptibilité dépend fortement de la cohérence du contexte. Plus récemment, Tang et al. avec le benchmark *ClashEval* (Wu *et al.*, 2024) ont proposé de quantifier ce « bras de fer » sur des tâches de questions-réponses en langage naturel. Ils ont confirmé les résultats de Xie et al. concernant l'importance de la cohérence du contexte sur le choix du LLM de se fier plutôt à sa mémoire paramétrique ou à sa mémoire contextuelle.

Notre étude prolonge ces travaux en explorant ce conflit dans un domaine concret : l'ingénierie logicielle et l'adaptation aux mises à jour d'APIs. Dans un environnement réel, les bibliothèques évoluent. Un LLM assistant de code a nécessairement été entraîné sur une version antérieure. S'il reçoit en contexte la documentation de la nouvelle version (en supposant un RAG parfait), le comportement attendu est qu'il privilégie *systématiquement* le contexte. Or, les LLMs ne sont pas entraînés pour accorder une confiance aveugle à leur contexte face à des connaissances massivement ancrées dans leurs poids, comme l'usage de la librairie `NumPy`.

Nos contributions sont les suivantes:

— **Développement d'un cadre d'évaluation fonctionnel** par exécution stricte du code généré

— **Introduction de métriques de dégradation** permettant d'isoler la perte de cohérence syntaxique de la perte d'intelligence

— **Quantification des pertes de performances** des modèles de la famille qwen3.5 sur deux perturbations différentes

2 Méthodologie et Cadre Expérimental

Notre but était donc de mesurer et d'évaluer à quel point les LLMs sont capables de s'adapter à des informations qui ne sont pas seulement nouvelles mais qui sont en conflit direct avec les données sur lesquelles ils ont été entraînés. Pour mesurer cette capacité d'adaptation et, le cas échéant, la perte de performance liée à cette nécessité d'adaptation nous avons tout d'abord généré des documentations contrefactuelles de trois types (ultra-minimale, minimale et explicative). Ensuite nous avons fourni cette documentation au LLM via le moteur d'inférence llama.cpp et le wrapper haut-niveau ollama. Nous avons fourni au LLM la documentation directement dans sa fenêtre de contexte (context window) en s'assurant que la taille de la documentation était inférieure à la taille maximale de la fenêtre de contexte habituelle du LLM. Nous avons utilisé le benchmark DS1000 introduit dans (Lai *et al.*, 2023) en tant que base de problèmes à résoudre en ne gardant que les problèmes liés à la librairie `NumPy` (ce qui fait un total de 159 problèmes à résoudre) et en modifiant les énoncés des problèmes pour s'assurer que la perturbation `NumPy` y était présente aussi. Ensuite nous avons récupéré le code généré par le LLM via des méthodes de parsing. Finalement, nous avons fait de l'évaluation fonctionnelle du code généré pour nous assurer de deux choses : Premièrement que le code utilise bien les modifications censées correspondre à une mise à jour d'une librairie. Deuxièmement que le code qui utilise les modifications idoines fonctionne correctement et renvoie les bonnes valeurs sur les différents test case de DS1000.

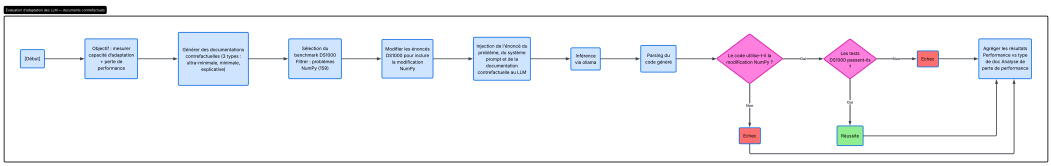

FIGURE 1 – Pipeline d'évaluation : de la conception de la documentation contrefactuelle à la double évaluation dynamique via wrappers et nettoyage AST.

2.1 Documentations Contrefactuelles et Perturbations

La génération de la documentation est déterministe. Nous évaluons trois niveaux de documentation : **Ultra-minimale** (catalogue des noms), **Minimale** (noms, arguments et signature), et **Explicative** (explicitation textuelle de la règle de perturbation).

Nous appliquons deux types de perturbations appliquées à l'attribut terminal :

— **Suffixe _v2 :** Ajout systématique de `_v2` (ex: `np.mean_v2`).

— **Capitale :** Mise en majuscule de la première lettre (ex: `np.Mean`).

2.2 Double Évaluation Fonctionnelle et Parsing AST

Contrairement aux évaluations textuelles classiques, notre pipeline valide le code par une *double exécution dynamique* :

1. **Évaluation avec injection (Wrapper) :** Le code est passé à un wrapper strict qui intercepte les accès à l'objet `np`. Si la convention perturbée n'est pas respectée, une `MissingSuffixError` est levée.
2. **Évaluation de contrôle :** Le même code, après suppression algorithmique des perturbations via nettoyage de l'Abstract Syntax Tree (AST), est exécuté avec l'import `numpy` standard.

Ainsi, le code est exécuté à la fois sur la librairie perturbée et sur la librairie originale. Cette méthode permet de vérifier si un code qui échoue à appliquer la perturbation était néanmoins algorithmiquement correct avec la version classique de `NumPy`.

3 Résultats et Analyse : La Famille Qwen3.5

Nous avons évalué la famille Qwen3.5 (0.8B, 2B, 4B, 9B, 27B, 35B) en inférence locale via ollama, en utilisant la métrique probabiliste $pass@10$.

3.1 Loi d'échelle et capacités de base

Sur les plis de contrôle (sans conflit), les performances croissent strictement avec le nombre de paramètres. Le taux de réussite passe de 15 % (0.8B) à 81 % (35B). Comme illustré sur la Figure 2, cette évolution semble suivre une relation logarithmique :

$$y = a \cdot \log(p) + b \tag{1}$$

FIGURE 2 – Performance de la famille Qwen3.5 sur les plis de contrôle en fonction du nombre de paramètres (gauche) et du logarithme du nombre de paramètres (droite).

3.2 Première analyse des performances sur les plis contrefactuels

Nous allons analyser dans cette partie les résultats des évaluations résumées dans la FIGURE 3

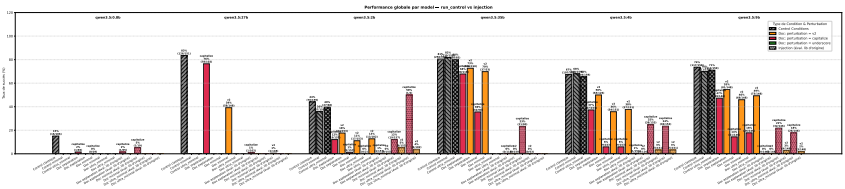

FIGURE 3 – Performances des modèles de la famille qwen3.5 sur les plis de contrôle (classique, documentation minimale et ultra minimale) et d'injections avec les perturbations _v2 et capitale (avec en contexte les documentations explicatives, minimales et ultra minimales) évalués sur la librairie d'origine (eval. lib d'origine) et la librairie contrefactuelle

Pour la fluidité des analyses que nous allons mener par la suite, il est important de noter que bien que qwen3.5:35b soit de la même famille que les autres modèles (même corpus d'entraînement, même structure de post-training) son architecture est différente des autres modèles puisqu'il se base sur une architecture Mixture-of-Experts (MoE) ce qui accélère l'inférence mais peut avoir un impact négatif sur la performance, notamment sur le raisonnement.

On constate aussi que les LLMs performent systématiquement mieux lors des plis de tests avec la documentation explicative plutôt qu'avec les documentations minimales ou ultra minimales. Tout se passe comme si le fait de devoir généraliser la perturbation provoquait une baisse de compétence du LLM. Ainsi, aider le LLM en lui expliquant la règle plutôt qu'en le laissant l'inférer sur un grand nombre d'exemples semblerait lui laisser moins de possibilités pour se tromper. C'est malheureux dans notre cas puisque nous souhaiterions que le LLM puisse réussir à utiliser seul la nouvelle documentation de la librairie.

Petite note sur l'interprétation du graphique 3 : Il pourrait arriver que la somme du pourcentage de réussite sur l'évaluation sur la librairie contrefactuelle et sur la librairie d'origine dépasse le pourcentage de réussite sur le pli de contrôle. En effet, comme l'évaluation est fonctionnelle et que certains problèmes peuvent être résolus sans utiliser la librairie `NumPy`, il arrive que certaines solutions soient comptabilisées comme correctes à la fois pour la librairie de base et la librairie contrefactuelle. Pour éviter ce cas de figure, nous avons décidé d'exclure les fonctions qui passent les deux évaluations.

3.3 Métriques de dégradation

On intoduit ici quelques définitions qui nous permettront par la suite de mener des analyses plus approfondies : Soient $\mathcal{E}_{ctrl}$ et $\mathcal{E}_{inj}$ les ensembles d'évaluations de contrôle et d'injection. On définit les sous-ensembles de succès :

$$
\begin{aligned}
S_{ctrl} &= \{e \in \mathcal{E}_{ctrl} \mid \text{control_passed}(e)\} \\
S_{inj} &= \{e \in \mathcal{E}_{inj} \mid \text{injection_passed}(e)\} \\
S_{inj_ctrl} &= \{e \in \mathcal{E}_{inj} \mid \text{control_passed}(e)\}
\end{aligned}
$$

où control_passed signifie que le code généré fonctionne quand il est executé avec la librairie originale, et injection_passed quand il fonctionne avec le wrapper. On remarquera que S_{ctrl_inj} n'est pas pris en

compte puisque l'on considère que cet ensemble est forcément vide (pourquoi le LLM générerait-t-il un code perturbé alors que la perturbation ne lui a même pas été précisée ?)

1. La perte de performance L_{perf} mesure la baisse de réussite stricte sur la tâche qui nous intéresse au final :

$$L_{perf} = \frac{|S_{ctrl}|}{|\mathcal{E}_{ctrl}|} - \frac{|S_{inj}|}{|\mathcal{E}_{inj}|} \tag{2}$$

2. La perte de compétence L_{comp} sanctionne l'incapacité à produire un code valide pour au moins l'une des deux librairies :

$$L_{comp} = \frac{|S_{ctrl}|}{|\mathcal{E}_{ctrl}|} - \frac{|S_{inj} \cup S_{inj_ctrl}|}{|\mathcal{E}_{inj}|} \tag{3}$$

3. La perte d'intelligence L_{int} s'appuie sur l'ensemble C des confusions (erreurs d'assertion) :

$$C = \{e \in \mathcal{E}_{inj} \mid e \notin (S_{inj} \cup S_{inj_ctrl}) \wedge (\text{stdout}(e) \in \text{Err}_{assert} \vee \text{stdout_control}(e) \in \text{Err}_{assert})\} \tag{4}$$

$$L_{int} = \frac{|C|}{|\mathcal{E}_{inj}|} \tag{5}$$

4. La perte de cohérence L_{coh} est définie comme le résidu de la perte de compétence :

$$L_{coh} = L_{comp} - L_{int} \tag{6}$$

3.4 Retour à l'analyse

On constate systématiquement (bien qu'a des degrés différents en fonction de la taille du modèle, du type de perturbation et du type de documentation) une perte de performance au sens où le LLM qui réussissait au préalable à résoudre un problème sur le pli de contrôle produit une solution qui ne s'exécute ni avec la librairie contrefactuelle, ni avec la librairie d'origine.

Néanmoins, nous pouvons constater que sur les plis explicatifs, le modèle perd moins en *compétence* que sur les plis minimal et ultra_minimal. En effet, la somme des évaluations sur la librairie contrefactuelle et la librairie d'origine donne, à peu de choses près, la pourcentage de réussite sur le pli de contrôle (81% de réussite sur le pli de contrôle pour qwen3.5:35b, 68% sur la perturbation capitale évaluée sur la librairie contrefactuelle et 0% évaluée sur la librairie originale ce qui donne une perte de performance de 12% voir 3). Ce phénomène se renforce à mesure que les modèles deviennent plus gros. Plus le modèle a de paramètres, moins la perte de *compétence* est grande sur la documentation explicative relativement au taux de succès de base.

Sur les documentations explicatives, les modèles augmentent leur *performance* (et leur *compétence*) d'abord en améliorant leur *coherence* (de qwen3.5:0.8b à qwen3.5:9b) puis en améliorant leur *intelligence* (qwen3.5:27b et qwen3.5:35b) comme on peut le voir sur la figure suivante.

On n'observe cependant pas le même comportement sur les plis de test s'effectuant avec les documentations minimales. En effet, dans ce cas c'est plutôt une amélioration linéaire de l'*intelligence* que l'on observe en fonction du logarithme du nombre de paramètres alors que la perte de *cohérence* est relativement stable en fonction du nombre de paramètres. On observe donc une mutation du régime d'amélioration des LLMs en fonction du nombre de paramètres en fonction du type de documentation

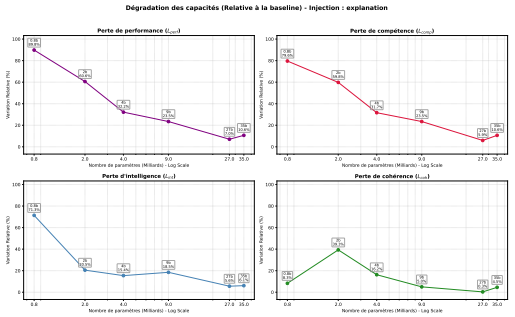

FIGURE 4 – Métriques de dégradation relatives au taux de succès (contrôle) pour les documentations **explicatives** (famille Qwen3.5).

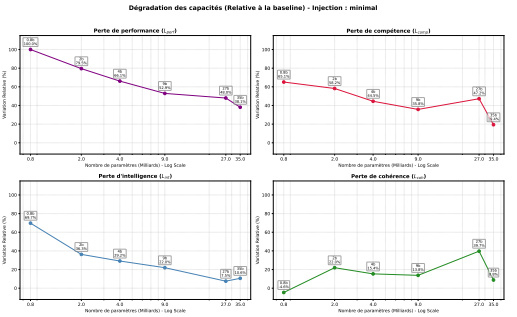

FIGURE 5 – Métriques de dégradation relatives au taux de succès (contrôle) pour les documentations **minimales** (famille Qwen3.5).

utilisée : explicative ou ressemblant à une vraie documentation. Dans le cas explicatif l'augmentation du nombre de paramètres fait d'abord majoritairement gagner de la cohérence puis il fait gagner de l'intelligence. Dans le cas minimal et ultra minimal on gagne constamment de l'intelligence mais la cohérence peine à suivre.

Remarquons sur ce graphique quelque chose d'étonnant : Le modèle qwen3.5:0.8b a une perte de cohérence négative. Cela peut s'expliquer par le fait que la perte d'intelligence est plus grande que la perte de compétence car là où le LLM échouait avant sur le pli de contrôle en faisant des erreurs de syntaxes ou de définitions de variables, il échoue maintenant en produisant un code qui s'éxécute correctement, mais qui bug. Ce comportement peut être plus attendu sur les modèles aux capacités de raisonnement faibles, comme c'est le cas par exemple ici de qwen3.5:0.8b. On peut expliquer ce phénomène d'une autre manière : Comme nous n'avons pas eu le temps de faire tourner suffisamment de tests sur le pli minimal capitalize de qwen3.5:0.8b nous n'avons pu l'évaluer que sur 19 problèmes là où il a été évalué sur 106 problèmes lors du pli de contrôle. Si les problèmes sur lesquels le modèle se trompe par intelligence sont plus concentrés parmi les premiers problèmes, cela biaise les résultats vers une perte d'intelligence plutôt qu'une perte de cohérence.

On peut aussi noter la légère perte d'intelligence et de cohérence et donc de compétence et de performance relative de qwen3.5:35b par rapport à qwen3.5:27b alors que ce dernier était moins bon sur les plis de contrôle. Cela peut s'expliquer par l'architecture MoE de qwen3.5:35b par rapport à celle dense de qwen3.5:27b

Finalement, il semble que les modèles de la famille qwen3.5 ait plus de facilité avec la perturbation _v2 qu'avec la perturbation capitale. Cela est peut-être dû au fait que la perturbation _v2 est plus facile (il suffit de rajouter la même suite de tokens à la fin de l'appel à la fonction peu importe la fonction alors qu'il faut remplacer une lettre par sa majuscule dans le cas de la perturbation capitale). Cela est peut-être aussi dû au fait que la perturbation _v2 semble plus plausible que la perturbation capitale ou a un mix des deux.

4 Limites de l'étude

Pour garantir un contrôle strict des variables expérimentales, toute notre documentation contrefactuelle a été générée algorithmiquement (par expressions régulières et parsing) à partir des *docstrings* existantes. Bien que cette méthode assure une reproductibilité parfaite, elle produit un texte très rigide. Dans la réalité, les développeurs font face à des notes de mise à jour (*release notes*), des guides de migration, ou des exemples de code annotés. Une piste complémentaire serait d'utiliser un LLM instructé pour réécrire et générer des documentations contrefactuelles plus naturelles et variées, plus proches de ce qu'un système RAG récupérerait en pratique. Nos modifications systématiques présentent un caractère inévitablement artificiel. Bien qu'une altération spécifique à chaque méthode refléterait mieux la réalité d'une mise à jour logicielle, nous avons privilégié une approche systématique pour des raisons d'automatisation. La conséquence directe de ce manque de plausibilité est une propension accrue du LLM à ignorer le contexte pour se replier sur sa mémoire paramétrique. En outre, notre but n'est pas de tester si le modèle sait appliquer une transformation syntaxique uniforme à toute une API — un exercice qui lui serait sans doute plus aisé mais déconnecté des conditions réelles de développement.

À l'instar des limites soulignées par Wu et al. dans *ClashEval* (Wu *et al.*, 2024), dont les questions reposaient sur des faits directs sans nécessiter de logique multi-étapes, notre évaluation sur le sous-ensemble NumPy de DS-1000 se concentre sur des snippets de code courts et isolés. L'intégration de ces informations conflictuelles dans des tâches d'ingénierie logicielle plus complexes (synthèse de documents multiples, modification à l'échelle d'un dépôt entier) reste une question ouverte.]

5 Potentielles solutions au problème initial

Pour résoudre le problème initial d'adaptation du LLM à une mise à jour d'API, nous proposons, en nous basant sur nos résultats, les potentielles pistes suivantes :

— Faire de la génération de code en se servant de deux instances du modèle. La première instance analyserait la documentation mise à jour et produirait une documentation explicative à la deuxième instance du LLM qui pourrait ensuite générer du code à partir de cette documentation explicative.

— Fine-tuner le modèle pour qu'il soit beaucoup plus receptif au contexte. Cette solution doit être envisagée avec prudence car si du contexte adversarial se retrouvait dans le contexte du LLM, celui-ci produirait alors presque certainement du code corrompu.

6 Conclusion

Ce travail s'est donné pour objectif de quantifier rigoureusement la capacité des grands modèles de langage à arbitrer entre leur mémoire paramétrique et une mémoire contextuelle contrefactuelle dans un cadre déterministe et d'évaluation fonctionnelle : la génération de code sous contrainte de mise à jour d'API. En transposant le paradigme de *ClashEval* (Wu *et al.*, 2024) jusqu'alors cantonné aux questions-réponses en langage naturel vers le domaine de l'ingénierie logicielle, nous avons proposé une pipeline d'évaluation originale reposant sur une double exécution dynamique (librairie

contrefactuelle via *wrapper* et librairie d'origine) et sur un nettoyage syntaxique par analyse d'arbre AST, garantissant que chaque verdict est d'ordre fonctionnel et non textuel.

6.1 Résultats importants.

Nos expériences permettent de dégager plusieurs conclusions :

Premièrement, aucun modèle testé ne parvient à basculer intégralement vers la mémoire contextuelle : même les meilleurs modèles (comme Qwen3.5:35b) affichent une perte de performance de l'ordre de 12 points sur la perturbation capitale lorsque la règle est explicitée, et de 44 points lorsqu'elle doit être inférée à partir d'une documentation implicite. La résistance de la mémoire paramétrique est un phénomène robuste, même s'il se résorbe, sans néanmoins s'annuler, avec la taille du modèle.

Deuxièmement, l'introduction de nos métriques de dégradation (L_{perf}, L_{comp}, L_{int}, L_{coh}) qui révèlent que l'adaptation du modèle emprunte deux trajectoires distinctes selon la nature de la documentation fournie. Avec une documentation *explicative*, la perte de compétence globale est minimisée. L'augmentation du nombre de paramètres du modèle opère une correction séquentielle: elle permet d'abord de stabiliser l'assimilation de la nouvelle convention (gain en cohérence), puis de restaurer la capacité de résolution algorithmique (gain en intelligence). À l'inverse, face à une documentation *implicite* (minimale ou ultra minimale), si l'intelligence croît de manière logarithmique avec la taille du modèle, la cohérence stagne face à la charge contextuelle. Le modèle échoue à maintenir la convention contrefactuelle, illustrant les limites d'un RAG fournissant des contextes non-explicatifs.

Troisièmement, dans le mode d'évaluation dit de contrôle (sans perturbation), nous ne remarquons pas de différences majeures de performance selon si nous fournissons aucune documentation, une documentation minimale ou une documentation ultra minimale. Nous pensions initialement que la taille du contexte aurait eu tendance à affaiblir les performance du modèle. Nous remarquons des résultats similaires dans le mode d'injection ou les documentations minimales et ultra minimales offrent des performances similaires.

Portée et originalité de la contribution. Au-delà des chiffres, une autre de nos contributions est méthodologique. En remplaçant la mesure de proximité sémantique utilisée dans *ClashEval* par une validation fonctionnelle stricte par exécution, nous proposons un protocole d'évaluation plus robuste. Cette *double exécution dynamique*, rendue possible par notre système de *wrappers* et de nettoyage AST, constitue une méthodologie réutilisable pour tout benchmark souhaitant évaluer l'arbitrage mémoire paramétrique/mémoire contextuelle dans un contexte de génération de code. Le cadre ainsi défini est générique : en écrivant de nouveaux wrappers, il est directement extensible à d'autres bibliothèques (Pandas, PyTorch, Scikit-learn), à d'autres types de perturbations (sémantiques, comportementales), ou à d'autres familles de modèles.

Le repo github *(en cours de nettoyage)* est disponible à l'adresse suivante : github

Références

LAI Y., LI C., WANG Y., ZHANG T., ZHONG R., ZETTLEMOYER L., YIH S. W.-T., FRIED D.,

WANG S. & YU T. (2023). Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning (ICML)*, p. 18319–18345: PMLR.

LEWIS P., PEREZ E., PIKTUS A., PETRONI F., KARPUKHIN V., GOYAL N., KÜTTLER H., LEWIS M., YIH W.-T., ROCKTÄSCHEL T. *et al.* (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, **33**, 9459–9474.

LONGPRE S., PERISETLA K., CHEN A., RAMESH N., DUBOIS C. & SINGH S. (2021). Entity-based knowledge conflicts in question answering. *arXiv preprint arXiv:2109.05052*.

WU K., WU E. & ZOU J. (2024). Clasheval: Quantifying the tug-of-war between an llm's internal prior and external evidence. *arXiv preprint arXiv:2404.10198*.

XIE J., ZHANG K., CHEN J., LOU R. & SU Y. (2023). Adaptive chameleon or stubborn sloth: Unraveling the behavior of large language models in knowledge conflicts. *arXiv preprint arXiv:2305.13300*.